



Migrer son code C/C++ en 64-bits

De nos jours, Windows est installé en 64 bits : on dit aussi mode x64 ou architecture amd64. Dans les fichiers projets, on nous propose de gérer cette cible x64 mais il y a de nombreux trucs et astuces à connaître lors du passage. Ce n'est pas si automatique que de juste recompiler les sources de vos applications et dll... Nous allons balayer les différents éléments avec le plus de détails possibles. Objectif : « Ça build ! »

Les outils nécessaires

Lorsque vous installez Visual Studio, vous avez le choix entre plusieurs SDK et plusieurs compilateurs. Actuellement, CL.EXE, le compilateur Microsoft est proposé en 4 versions :

- cl.exe 32 bit pour X86,
- cl.exe 64 bit pour X64,
- cl.exe 32 bit et 64 bit pour ARM.

Les grands changements

Le type int et long sont des valeurs 32-bit. Le type size_t est une valeur 64-bit. Voici le détail :

Scalar Type	C Data Type	Storage Size (in bytes)	Recommended Alignment
INT8	char	1	Byte
UINT8	unsigned char	1	Byte
INT16	short	2	Word
UINT16	unsigned short	2	Word
INT32	int, long	4	Dword
UINT32	unsigned int, unsigned long	4	Dword
INT64	__int64	8	Quadword
UINT64	unsigned __int64	8	Quadword
FP32 (single precision)	float	4	Dword
FP64 (double precision)	double	8	Quadword
POINTER	void*	8	Quadword
__m64	struct __m64	8	Quadword
__m128	struct __m128	16	Octaword

La gestion des chaînes de caractères est plus délicate. Faisons un petit rappel sur les définitions Windows. Il existe CHAR, WCHAR, LPSTR, LPCSTR, LPTSTR, LPCTSTR. Voyons les définitions Windows et leurs équivalents que votre projet supporte l'Unicode ou pas.

// Generic types

```
#ifndef UNICODE
typedef wchar_t TCHAR;
#else
typedef unsigned char TCHAR;
#endif
```

typedef TCHAR *LPTSTR, *LPCTSTR;

// 8-bit character specific

```
typedef unsigned char CHAR;
typedef CHAR *LPSTR, *LPCH;
```

// Unicode specific (wide characters)

```
typedef unsigned wchar_t WCHAR;
typedef WCHAR *LPWSTR, *LPWCH;
```

Si le projet supporte Unicode, la gestion des chaînes est assurée via wchar_t pour TCHAR. Ce qu'il faut retenir de ces déclarations c'est que TCHAR est un allié utile. En effet, TCHAR via tchar.h va nous permettre de faire un code générique dans la gestion des chaînes de caractères. Pour déclarer une chaîne, on utilise la définition TCHAR :

TCHAR sz[255];

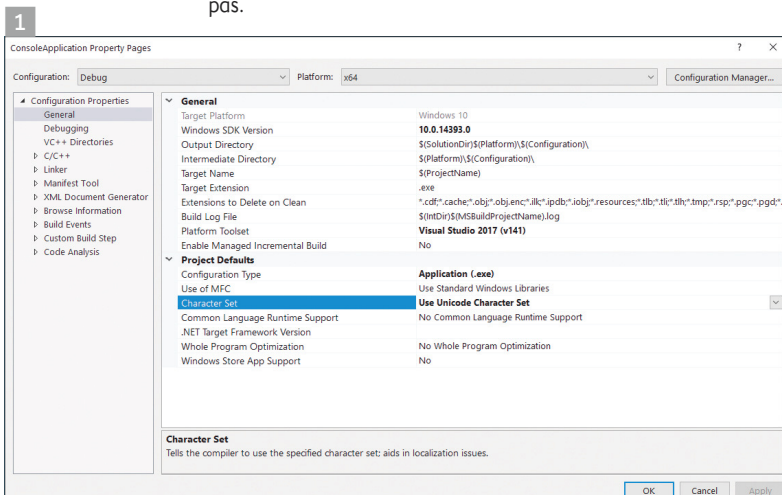
TCHAR vaudra soit char ou wchar_t. Comment le savoir ? Tout dépend si votre projet est Unicode ! Si vous êtes en « Character Set » = « Use Unicode Character Set » alors votre projet va définir la constante UNICODE au niveau du préprocesseur. Regardons les propriétés du projet où cela est défini : 1

Pour être transparent, je vous engage à coder de manière multiplatforme. Pour cela, il faut une table de correspondance de routines C. <https://msdn.microsoft.com/en-us/library/tsbaswba.aspx>

En voici un extrait : 2

Si vous faites du TCHAR, vous utilisez la routine générique. Et suivant si UNICODE est définie ou pas, le nom de la fonction est étendu en nom voulu. Exemple de code à migrer :

```
char sz[255];
sprintf(sz, "Elapsed ms since Windows is up: %ld ms\n", GetTickCount());
```



```
printf(sz);
```

Pour que ce code compile en x86 et en x64, il faut prendre des précautions et obtenir la correspondance de `sprintf` et `printf` en mode TCHAR.h :

Generic-Text Routine Mappings

TCHAR.H routine	_UNICODE & _MBCS not defined	_MBCS defined	_UNICODE defined
<code>_sprintf</code>	<code>sprintf</code>	<code>sprintf</code>	<code>_swprintf</code>
<code>_sprintf_l</code>	<code>_sprintf_l</code>	<code>_sprintf_l</code>	<code>_swprintf_l</code>

Generic-Text Routine Mappings

TCHAR.H routine	_UNICODE & _MBCS not defined	_MBCS defined	_unicode defined
<code>_printf</code>	<code>printf</code>	<code>printf</code>	<code>wprintf</code>

Ainsi, nous savons quels sont les noms des fonctions à employer :

```
#include <tchar.h>
```

```
TCHAR sz2[255];
_sprintf(sz2, _T("Elapsed ms since Windows is up: %ld ms\n"), GetTickCount());
_tprintf(sz2);
```

Comme vous pouvez le constater, les chaînes sont bordées par `_T(" ")`. En mode Not Set ou MBCS, `_T` ne vaut rien alors qu'en UNICODE, il vaut L. C'est comme ça que l'on désigne une chaîne Unicode.

Autre solution ISO : la STL

Il est possible de gérer le type chaîne de caractères via la STL du C++ : la Standard Template Library. Il suffit de faire `#include <string>` et de déclarer soit des `std::string` ou des `std::wstring`. Ce sont soit des chaînes de type `char` ou de type `wchar_t`. Tout dépend si on veut coder pour Windows ou multiplateforme. Pour être complètement ISO, je vous engage à utiliser cette solution à base de STL. Le standard ISO, c'est le langage et la STL. Sachez qu'entre experts, nous n'avons pas tous le même point de vue. Mon pote Alain Z, de Redmond recommande de coder en `wchar_t`. En fait, moi je vois deux modes. Suivant si on fait une dll ou suivant si on fait un exe, suivant si on attaque des API Windows, suivant si on fait du Boost, un ensemble de librairies C++ très connues disponible sur Boost.org. Si on fait du Boost, on va se retrouver à faire du `string` ou du `wstring` à tous les étages (threads, filesystem, regexp, etc). Voici ce que cela donne avec la STL :

```
#include <string>
#include <iostream>
```

Generic-text routine name	SBBCS (_UNICODE & _MBCS not defined)	_MBCS defined	_UNICODE defined
<code>_cgetts</code>	<code>_cgets</code>	<code>_cgets</code>	<code>_cgetws</code>
<code>_cgetts_s</code>	<code>_cgets_s</code>	<code>_cgets_s</code>	<code>_cgetws_s</code>
<code>_cputts</code>	<code>_cputs</code>	<code>_cputs</code>	<code>_cputws</code>
<code>_fgettc</code>	<code>fgetc</code>	<code>fgetc</code>	<code>fgetc</code>
<code>_fgettcchar</code>	<code>_fgetcchar</code>	<code>_fgetcchar</code>	<code>_fgetcchar</code>
<code>_fgetts</code>	<code>fgets</code>	<code>fgets</code>	<code>fgetws</code>
<code>_fputtc</code>	<code>fputc</code>	<code>fputc</code>	<code>fputc</code>
<code>_fputtcchar</code>	<code>_fputcchar</code>	<code>_fputcchar</code>	<code>_fputcchar</code>
<code>_fputts</code>	<code>fputs</code>	<code>fputs</code>	<code>fputws</code>
<code>_ftprintf</code>	<code>fprintf</code>	<code>fprintf</code>	<code>fuprintf</code>

```
std::wstring sz3 = _T("hello world!");
std::wcout << sz3 << std::endl;
```

Les conversions std::string & std::wstring

Si vous utilisez les API Windows, vous serez amené à utiliser TCHAR. Si vous avez du code externe, par exemple, qui utilise la STL et `std::wstring` il va falloir faire des correspondances. Comment fait-on ? Avec la STL ce n'est pas difficile ; on peut revenir au type de base via la méthode `c_str()` :

```
std::wstring s1 = L"UNICODE direct";
const wchar_t* ws1 = s1.c_str();
```

```
std::string s2 = "Hello world";
const char* ss2 = s2.c_str();
```

En utilisant le ctor de `wstring` on peut passer d'un `string` à `wstring` facilement et vice-versa :

```
std::string a1 = "A1";
std::wstring u1 = _T("U1");

std::wstring u2(a1.begin(), a1.end());
std::wcout << u2 << std::endl;
```

```
std::string a2(u1.begin(), u1.end());
std::cout << a2 << std::endl;
```

Ce genre d'opérations n'est pas coûteux. Pour évoluer dans une situation confortable, je choisis la STL et je fais des allers-retours de temps en temps avec les TCHAR quand il faut faire des appels avec les API Windows. Si je fais une GUI en MFC, j'utilise les classes CString. Vous allez me dire : « mais c'est le "bordeau", on utilise tout ??? ». Vous avez raison mais voici mon explication. Certaines API prennent des types ; si on passe son temps à convertir, on ne s'en sort plus... C'est déjà assez complexe par exemple quand en COM on doit manipuler des BSTR ou des VARIANTS... Bref, utilisez ce qui est naturel.

Autre solution Windows via les MFC et ATL/MFC

Il est aussi possible de prendre une classe générique CString gérée par la partie commune ATL/MFC. Cette classe CString existe en plusieurs variantes : `CStringT`, `CStringA`, `CStringW`. Simple à comprendre si vous avez lu l'article depuis le départ... Au milieu de ce code vous pouvez voir les fonctions A2T et T2A qui sont bien pratiques. Pour les utiliser, il faut déclarer la macro `USE_CONVERSION` juste derrière le { d'entrée dans la fonction.

```
CString cs1 = sz;
CString cs2 = s1.c_str();
CString cs3 = A2T(s2.c_str());
CStringW cs4 = sz;
CStringW cs5 = s1.c_str();
CStringA cs6 = s2.c_str();
CStringA cs7 = T2A(s1.c_str());
```

Il existe des variantes :

- A2T et T2A
- W2T et T2W
- A2W et W2A

Compiler directive in effect	T becomes	OLE becomes
none	A	W
_UNICODE	W	W
OLE2ANSI	A	A
_UNICODE and OLE2ANSI	W	A

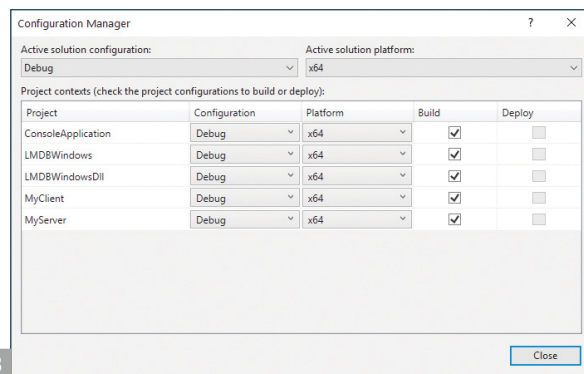
Dans le nom des macros, le type de chaînes sources est à gauche et le type de chaînes destinations est à droite. A veut dire LPSTR, T pour LPTSTR et W pour LPWSTR.

Le projet C++ x64 type

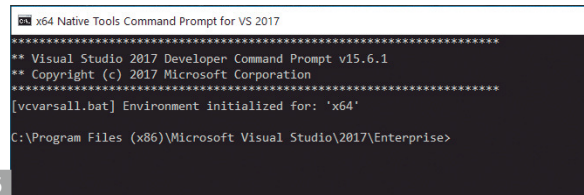
Tout se passe dans la boîte de paramétrages du projet. Il faut définir une plateforme x64 via le Configuration Manager si elle n'existe pas... **3**

Nommage des modules

Il faut une certaine rigueur dans le nommage des modules. Boost est un modèle à suivre.



3



5

On connaît les informations suivantes rien qu'au nom du module :

- Est 32 bit ou 64 bit,
- S'il est debug ou pas,
- S'il est shared ou static,
- La version du Visual C++ avec lequel il est compilé,
- La version de la librairie. **4**

Rappel sur les dépendances

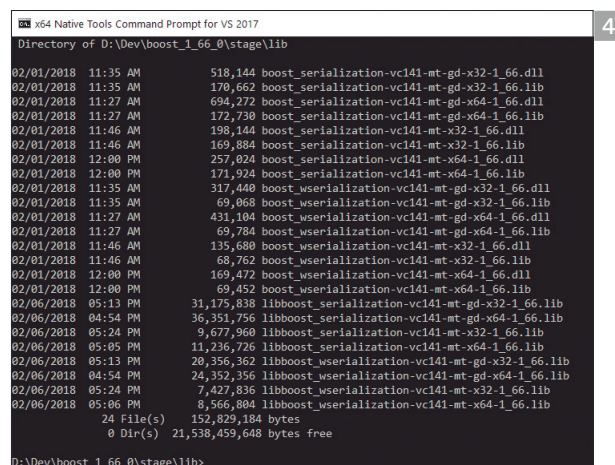
Les modules doivent tous être compilés en x64. On ne mixe pas x86 et x64 sinon ça plante ! On build soit en release soit en debug mais on ne mixe pas !

Compilation en ligne de commandes

Il faut lancer le raccourci « x64 Native Tools Command Prompt for VS 2017 » et voilà : **5**

Conclusion

Le passage en 64-bit via la recompilation des sources peut-être un travail coûteux et qui prend du temps. Tout dépend de la qualité du code. La gestion des chaînes de caractères peut-être le frein à ce passage. Avec les règles décrites dans cet article, on possède les trucs et astuces pour s'en sortir sans trop forcer.



4

reste cadre de 185 mm de large x 65 mm de haut